

# Interview Cheatsheet

Fraud Detection Project — Every Topic at a Glance  
Revise this the night before your interview

---

10 topics · Key formulas · Key code · Common mistakes · One-line answers

## TOPIC 1 — Class Imbalance

### The Problem

99.83% legit, 0.17% fraud → model always says legit

Accuracy = 99.83%, Recall = 0% → useless

### Key Numbers

Dataset: 284,807 transactions

Fraud: 492 (0.17%)

Imbalance ratio: 578:1

### Fix 1: SMOTE (for LR)

```
smote = SMOTE(random_state=42, k_neighbors=5)
```

```
X_sm, y_sm = smote.fit_resample(X_train, y_train)
```

Rule: AFTER split, ONLY on X\_train

### SMOTE internals

1. Pick fraud sample
2. Find 5 nearest fraud neighbours
3. New sample = original + rand(0,1)\*(neighbour-original)
4. Repeat until balanced

### Fix 2: scale\_pos\_weight (for XGBoost)

```
scale = neg / pos # 284315 / 492 = 578
```

```
XGBClassifier(scale_pos_weight=scale)
```

Do NOT combine SMOTE + scale\_pos\_weight

### class\_weight='balanced'

```
weight_i = n_samples / (n_classes * n_i)
```

fraud weight ≈ 578

For LogisticRegression — one-line fix

---

## TOPIC 2 — Evaluation Metrics

### The 4 Confusion Matrix Cells

TP: fraud caught ✓ (money saved)

FP: legit blocked ✗ (angry customer)

FN: fraud missed ✗✗ (money lost — WORST)

TN: legit passed ✓ (normal operation)

### Key Formulas

Precision =  $TP / (TP + FP)$

Recall =  $TP / (TP + FN)$

F1 =  $2 * P * R / (P + R)$

F2 =  $5 * P * R / (4 * P + R)$

MCC =  $(TP * TN - FP * FN) / \text{sqrt}(...)$

### Which metric to use

AUC-PR: primary (honest for imbalance)

Recall: catch rate — primary business metric

Precision: alert quality

F2: when recall > precision in importance

NOT accuracy, NOT AUC-ROC (misleading)

### Threshold Tuning

```
precisions, recalls, thresholds =  
precision_recall_curve(y_test, y_prob)
```

```
f2 = (5 * P * R) / (4 * P + R + 1e-9)
```

```
best_thresh = thresholds[  
    argmax(f2[: -1])]
```

Default 0.5 → usually wrong for fraud

F2 optimal → typically 0.25-0.40

## TOPIC 3 — Models: LR vs XGBoost

### Logistic Regression (Baseline)

```
lr = LogisticRegression(  
class_weight='balanced',  
max_iter=1000, solver='lbfgs')  
lr.fit(X_train, y_train)
```

When LR beats XGBoost:

Small dataset (<5k), linear patterns,  
constrained hardware, strict interpretability

### Key LR parameters

class\_weight: adjusts loss function weights  
max\_iter: increase from default 100 → 1000  
solver: lbfgs for medium datasets

### XGBoost (Production)

```
xgb = XGBClassifier(  
n_estimators=1000,  
max_depth=6, learning_rate=0.05,  
subsample=0.8, colsample_bytree=0.8,  
scale_pos_weight=578,  
early_stopping_rounds=50)
```

### Key XGBoost parameters

n\_estimators: set high, let early stopping decide  
max\_depth: 4-8 for fraud (overfitting risk)  
subsample: row-level regularisation  
colsample\_bytree: column regularisation  
early\_stopping\_rounds: prevents overfitting

## TOPIC 4 — FastAPI Endpoint

### Startup pattern (load model ONCE)

```
model_store = {}  
  
@asynccontextmanager  
async def lifespan(app):  
    model_store['model'] = joblib.load(...)  
    yield # serve requests  
    model_store.clear()
```

### Request validation

```
class TransactionRequest(BaseModel):  
    transaction_id: str  
    features: list[float]  
  
    @validator('features')  
    def check_len(cls, v):  
        if len(v) != 30: raise ValueError
```

### Predict endpoint

```
@app.post('/predict')  
def predict(tx: TransactionRequest):  
    X = np.array(tx.features).reshape(1, -1)  
    prob = model.predict_proba(X)[0, 1]  
    is_fraud = prob >= threshold  
    return {'fraud_probability': prob, ...}
```

### Key rules

- sync def (not async): CPU-bound inference
- async def: only for I/O-bound (DB, HTTP)
- Load model at startup, NOT per request
- transform() not fit\_transform() for scalars
- Return float() not np.float32 (JSON-safe)

### Endpoints

- POST /predict → fraud decision
- GET /health → Kubernetes health check
- GET /docs → auto OpenAPI documentation

---

## TOPIC 5 — Docker

### Dockerfile (correct order)

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
COPY models/ models/
```

```
RUN useradd -m appuser && USER appuser
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app:app",
```

```
"--host", "0.0.0.0", "--port", "8000"]
```

### Why this order matters

requirements.txt first → Docker caches pip layer

Code last → code changes don't invalidate pip cache

### Commands

```
docker build -t fraud-api:v1 .
```

```
docker run -p 8000:8000 fraud-api:v1
```

```
docker ps # list running
```

```
docker logs # view logs
```

```
docker stop
```

### Key rules

python:3.11-slim not full (saves ~850MB)

Run as non-root user (security)

--no-cache-dir (saves ~50MB)

EXPOSE documents, does NOT open port

Never COPY .csv, .env, notebooks

## TOPIC 6 — CI/CD with GitHub Actions

### Workflow file location

```
.github/workflows/ci-cd.yml
```

### Trigger

```
on:
```

```
push:
```

```
branches: [main]
```

```
pull_request:
```

```
branches: [main]
```

### Test job

```
jobs:
```

```
test:
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- uses: actions/checkout@v4
```

```
- uses: actions/setup-python@v5
```

```
- run: pip install -r requirements.txt
```

```
- run: pytest tests/ -v
```

### Build job (needs test to pass)

```
build:
```

```
needs: test
```

```
steps:
```

```
- run: docker build -t fraud-api .
```

```
- uses: docker/login-action@v3
```

```
- run: docker push username/fraud-api
```

### Secrets (never hardcode)

GitHub → Settings → Secrets → Actions

DOCKER\_USERNAME, DOCKER\_PASSWORD

Referenced: `\${{ secrets.DOCKER\_USERNAME }}

### What CI/CD gives you

Automated tests on every push

Green checkmark badge on GitHub repo

Auto build Docker image on merge

Auto deploy to Render on merge

Interviewers notice this immediately

## TOPIC 7 — MLflow Experiment Tracking

### Log a run

```
mlflow.set_experiment('fraud-detection')
```

```
with mlflow.start_run(run_name='xgb-v1'):
```

```
mlflow.log_param('n_estimators', 300)
```

```
mlflow.log_metric('auc_pr', 0.89)
```

```
mlflow.xgboost.log_model(xgb, 'model')
```

```
mlflow.log_artifact('pr_curve.png')
```

### View UI

```
mlflow ui → http://localhost:5000
```

### Load model

```
model = mlflow.xgboost.load_model(
```

```
'models:/fraud-detector/Production')
```

### param vs metric

log\_param: set before training (n\_estimators)

log\_metric: measured after training (recall)

### Model registry stages

None → Staging → Production → Archived

Change stage in UI or programmatically

Rollback: transition old version back to Production

### What to log

Params: all hyperparams, threshold, beta

Metrics: AUC-PR, recall, precision, savings

Artifacts: model.pkl, scalars, plots, requirements.txt

### Key benefit

Compare 10 runs side by side

Reproduce any historical result

Safe rollback when new model fails

---



## TOPIC 8 — SHAP Explainability

### Setup

```
explainer = shap.TreeExplainer(xgb)
```

```
shap_values = explainer.shap_values(X_test)
```

### Global importance (bar plot)

```
shap.summary_plot(shap_values, X_test,  
plot_type='bar')
```

### Beeswarm (direction + importance)

```
shap.summary_plot(shap_values, X_test)
```

Red=high feature value, Blue=low

Right=pushed toward fraud

### Waterfall (one transaction)

```
shap.plots.waterfall(shap.Explanation(  
values=shap_values[i],  
base_values=explainer.expected_value,  
data=X_test.iloc[i]))
```

### SHAP value intuition

Base = average model output (fraud rate ~0.17%)

SHAP value = how much THIS feature pushed

this prediction above/below base

Sum(SHAP) + base = predicted probability

### TreeExplainer vs KernelExplainer

TreeExplainer: fast, exact, tree-specific

KernelExplainer: model-agnostic, slow

Always use TreeExplainer for XGBoost

### Why SHAP matters

GDPR Article 22: must explain automated decisions

Fraud analyst reads: 'V14 unusual, Amount 7x avg'

Legally auditable explanation

### Interview one-liner

'SHAP gives each feature a signed contribution

showing why a specific transaction was flagged'

---

## TOPIC 9 — GitHub Deployment Step by Step

### Project structure

fraud-detection/

data/.gitkeep ← empty (never commit CSV)

src/train.py

api/app.py

models/.gitkeep ← empty (never commit.pkl)

tests/test\_api.py

Dockerfile

requirements.txt

.gitignore

README.md

### .gitignore must include

\*.pkl \*.csv .env mlruns/ \_\_pycache\_\_/

### Push to GitHub

```
git init
```

```
git add .
```

```
git commit -m 'Initial commit'
```

```
git remote add origin https://github.com/...
```

```
git branch -M main
```

```
git push -u origin main
```

### Free cloud deploy (Render.com)

1. render.com → New Web Service
2. Connect GitHub repo
3. Select Docker as runtime
4. Add environment variable: MODEL\_DIR=models
5. Deploy → get public URL

### Result

Live URL: fraud-api.onrender.com/docs

Demo live in interview!

---

## TOPIC 10 — 10 Common Mistakes to Never Make

### Mistake 1: SMOTE before train/test split

→ Synthetic test samples in training. Evaluation is dishonest. All metrics are inflated.

### Mistake 2: SMOTE + scale\_pos\_weight together

→ Double-correction. Model too aggressive. Precision drops below 50%.

### Mistake 3: One scaler for both Amount and Time

→ Second fit\_transform overwrites first. Saved scaler has wrong stats.

### Mistake 4: Loading model per request in FastAPI

→ 500ms disk read per call. SLA is 100ms total. Every request times out.

### Mistake 5: Using transform() before fit()

→ ValueError: scaler not fitted. Always fit on training data first.

### Mistake 6: Evaluating with accuracy on imbalanced data

→ 99.83% accuracy, 0% fraud recall. Accuracy is meaningless here.

### Mistake 7: Applying correlation analysis on full dataset before split

→ Subtle leakage. Feature selection biased toward test patterns.

### Mistake 8: Committing model .pkl files to git

→ Large files slow repo. Use MLflow artifacts + joblib for model storage.

### Mistake 9: Using async def for CPU-bound inference in FastAPI

→ Blocks the event loop. sync def runs in thread pool — correct for ML.

### Mistake 10: Not pinning versions in requirements.txt

→ pip installs latest → breaking changes in CI. Always pin: xgboost==2.0.3